LECTURE 4

TRUSTWORTHINESS, COGNITIVE ARCHITECTURES, LANGUAGE AGENTS

JANUARY 14, 2026

DATASCI290: NEUROSYMBOLIC AI

PRODUCTION SYSTEMS: THE CORE ENGINE

- A **production system** is
  - A set of IF–THEN rules operating over a working memory
  - IF matches current state; THEN proposes or executes an action
  - For instance
    - XYZ —> XWZ
    - W —> AB
    - ….
- Why production systems were revolutionary
  - Unified representation of knowledge and control
  - Behavior emerges from rule interaction, not fixed programs
- Key mechanisms
  - Pattern matching: which rules apply now?
  - Conflict resolution: which rule fires?
  - Execution: modify memory or act in the world

THERMOSTAT EXAMPLE

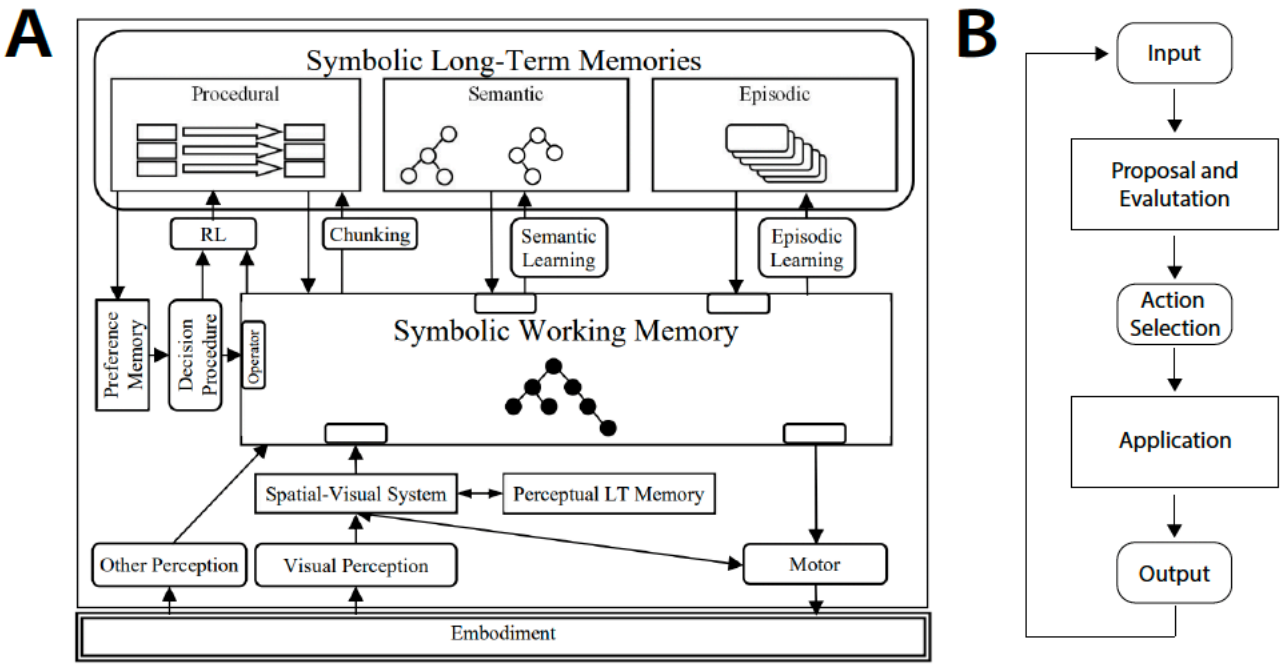
$$
\begin{aligned}
(\text{temperature} > 70^\circ) \wedge (\text{temperature} < 72^\circ) &\rightarrow \text{stop} \\
\text{temperature} < 32^\circ &\rightarrow \text{call for repairs; turn on electric heater} \\
(\text{temperature} < 70^\circ) \wedge (\text{furnace off}) &\rightarrow \text{turn on furnace} \\
(\text{temperature} > 72^\circ) \wedge (\text{furnace on}) &\rightarrow \text{turn off furnace}
\end{aligned}
$$

- Implications
  - Observe temperature → apply rule → turn heater on/off
  - Demonstrates perception, rule application, and action in a loop
- Essentially
  - Even trivial agents require state, rules, and control
  - Scaling up adds complexity, not new kinds of logic

FROM PRODUCTION SYSTEMS TO COGNITIVE ARCHITECTURES

- Why production systems were not enough
  - No built-in perception, learning, or long-term memory
  - Difficult to scale to complex, real-world tasks
- Cognitive architectures added structure
  - Working memory + long-term memory
  - Explicit decision procedures
  - Learning mechanisms
- Examples
  - Soar, ACT-R, and others
  - Designed to model human cognition and build complete agents

SOAR



- Laird, John E., and Paul S. Rosenbloom. "The evolution of the Soar cognitive architecture." In *Mind matters*, pp. 1-50. Psychology Press, 2014.

CONTROL FLOW AS THE HEART OF COGNITION

- Why control flow matters
  - Determines when to reason, when to act, when to learn
  - Separates random behavior from purposeful behavior
- Control in production systems
  - Rule matching and conflict resolution implement control
  - State of working memory guides execution path
- Control in agents
  - Task decomposition
  - Goal management
  - Subgoal creation and resolution

SOAR

- Memory
- Grounding
- Decision making
- Learning

ENTER LANGUAGE MODELS – WHAT CHANGED

- What LLMs provide
  - Powerful pattern recognition over language
  - Implicit knowledge learned from massive data
  - Ability to generate plans, explanations, and code
- What LLMs do not provide
  - Explicit control flow
  - Guaranteed correctness
  - Stable long-term memory
  - Hard constraints

LANGUAGE MODELS AS PROBABILISTIC PRODUCTION SYSTEMS

- The core analogy
  - Production rule: IF state THEN action
  - LLM: given prompt (state), sample next token/action
  - Both map state to action
- Why 'probabilistic' matters
  - Multiple possible actions with different probabilities
  - No guarantee the 'right' rule fires
  - Repeatability is not assured
- Implication
  - Great flexibility
  - Poor reliability for high-consequence decisions

PROMPT ENGINEERING AS CONTROL FLOW

- Prompts shape behavior
  - What you include determines what the model attends to
  - Few-shot examples bias rule selection
  - Instructions act as soft constraints
- Chaining prompts
  - Implements multi-step reasoning
  - Each prompt-response pair is a production step
- Why this is fragile
  - No hard guarantees
  - Small wording changes can change behavior

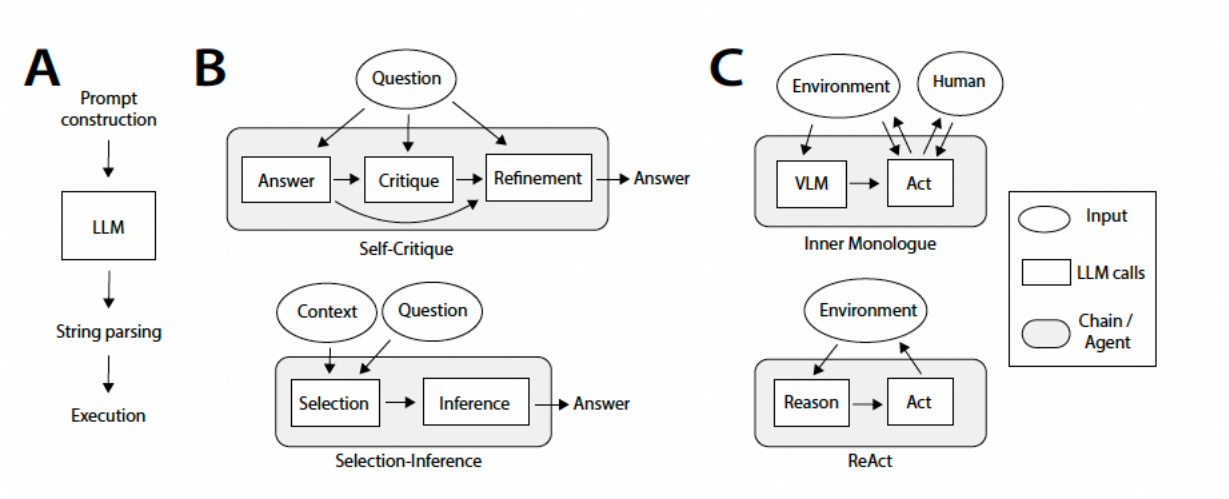
FROM SINGLE CALLS TO AGENT LOOPS

- Single LLM call
  - Input → output
  - No memory, no environment interaction
- Agent loop
  - Observe → think → act → observe → repeat
  - State persists across steps
- Why loops matter
  - Enables planning, correction, and adaptation
  - Moves from answering to acting

PROMPTING METHODS AND PRODUCTION SEQUENCES

Prompting Method	Production Sequence
Zero-shot	$Q \overset{\text{LLM}}{\rightsquigarrow} Q\ A$
Few-shot	$Q \longrightarrow Q_1\ A_1\ Q_2\ A_2\ Q \overset{\text{LLM}}{\rightsquigarrow} Q_1\ A_1\ Q_2\ A_2\ Q\ A$
Retrieval Augmented Generation	$Q \overset{\text{Wiki}}{\longrightarrow} Q\,O \overset{\text{LLM}}{\rightsquigarrow} Q\,O\,A$
Socratic Models	$Q \overset{\text{VLM}}{\rightsquigarrow} Q\,O \overset{\text{LLM}}{\rightsquigarrow} Q\,O\,A$
Self-Critique	$Q \overset{\text{LLM}}{\rightsquigarrow} Q\,A \overset{\text{LLM}}{\rightsquigarrow} Q\,A\,C \overset{\text{LLM}}{\rightsquigarrow} Q\,A\,C\,A$

PROMPTING METHOD, AND PRODUCTION SEQUENCES



- Essentially, modern LLM-based systems are gradually reconstructing classical cognitive control structures
  - Using prompts, chains, and agent loops
- Types
  - A = primitive, linear execution
  - B = internal reasoning loops
  - C = full agent interaction with environment

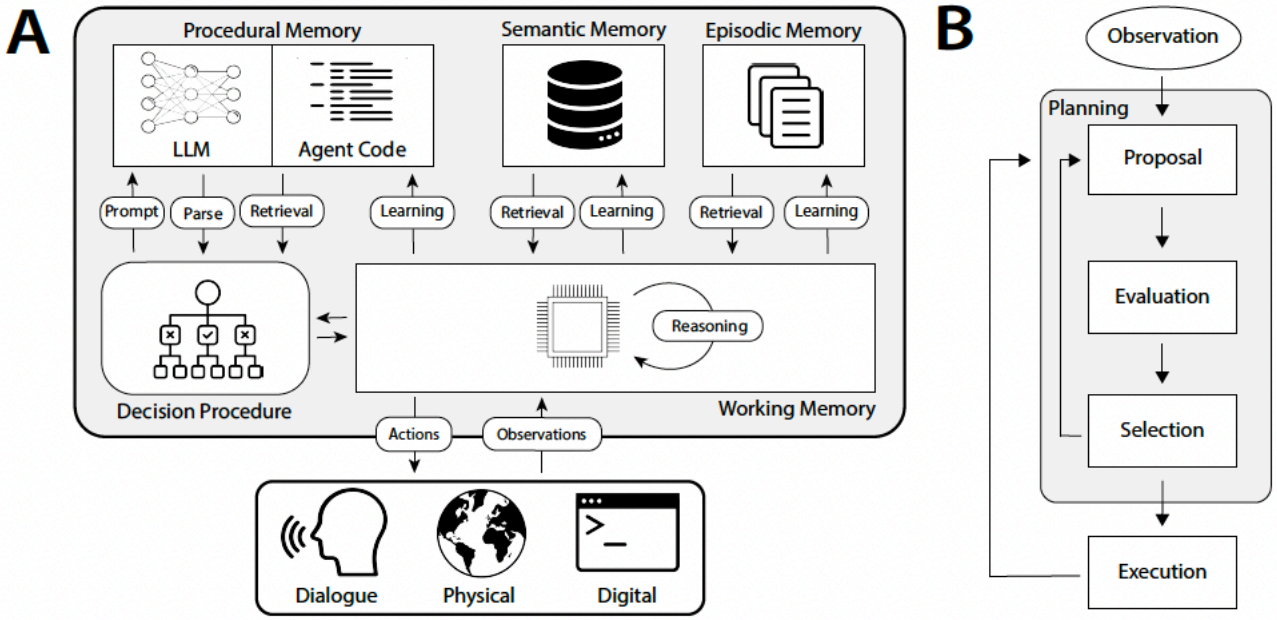
WHY 'COGNITIVE' AGENTS ARE DIFFERENT

- Reactive agents
  - Respond to observations
  - Little internal structure
- Cognitive agents
  - Have internal goals, plans, and memories
  - Reason about their own state
  - Learn from experience
- Key shift
  - From stimulus-response to deliberation

COALA – THE ARCHITECTURAL BLUEPRINT

- What CoALA proposes
  - A modular architecture for cognitive language agents
  - Explicit memory systems
  - Explicit action types
  - Explicit decision procedure
- Why an architecture is needed
  - To make behavior predictable
  - To enable inspection and control
  - To support learning over time

COALA: AGENT ARCHITECTURE



MEMORY IN COALA

- Working memory
  - Current task state
  - Goals, constraints, intermediate results
- Semantic memory
  - Facts, concepts, domain knowledge
- Episodic memory
  - Specific past experiences
- Procedural memory
  - Skills, routines, code, methods

WHY MEMORY IS CENTRAL

- Without memory
  - Agents forget past actions
  - Cannot build experience
  - Cannot improve over time
- With memory
  - Agents accumulate knowledge
  - Agents develop skills
  - Agents become more reliable

ACTION TYPES IN COALA

- Internal actions
  - Reasoning
  - Retrieval
  - Learning
- External actions
  - Tool use
  - API calls
  - Physical actions (real or simulated)
- Why the distinction matters
  - Internal actions change the agent
  - External actions change the world

GROUNDING – CONNECTING TO REALITY

- What grounding is
  - Linking symbols to real-world data or actions
  - APIs, sensors, databases, humans
- Why grounding is critical
  - Prevents hallucination
  - Enables verification
  - Provides feedback

RETRIEVAL – ACCESSING KNOWLEDGE

- Retrieval as an action
  - Agent decides when and what to retrieve
  - Not automatic; part of control flow
- Sources
  - Semantic memory
  - Episodic memory
  - External stores (databases, KGs, documents)

REASONING – BUILDING INTERNAL STRUCTURE

- What reasoning actions do
  - Decompose tasks
  - Generate candidate plans
  - Critique options
- Why reasoning alone is insufficient
  - Still probabilistic
  - Can be wrong with confidence

LEARNING – CHANGING THE AGENT

- Semantic learning
  - Adding new facts or correcting beliefs
- Episodic learning
  - Storing experiences
- Procedural learning
  - Acquiring new skills or routines
- Risk
  - Bad learning hardens errors

DECISION CYCLE: PROPOSE, EVALUATE, SELECT, EXECUTE

- Propose
  - Generate possible actions
- Evaluate
  - Estimate value, risk, or utility
- Select
  - Choose one action
- Execute
  - Perform action and update state

EXPLICIT DECISION STRUCTURE MATTERS

- Debuggability
  - We know where errors enter
- Safety
  - We can insert checks before execution
- Governance
  - We can audit decisions

COALA: AGENT EXAMPLES

- ReAct
  - Reasoning + acting loop
  - Minimal memory
- Voyager
  - Procedural memory growth (code)
- Generative Agents
  - Episodic + semantic memory

WHAT COALA SOLVES

- Brings structure back
  - Explicit memory
  - Explicit control
  - Explicit decision cycle
- Unifies classical AI and modern LLMs
  - Rules + learning
  - Symbols + neural models

WHAT COALA DOES NOT SOLVE

- Hard guarantees
  - Still probabilistic
- Formal verification
  - Needs symbolic layers
- Ethics and policy
  - Must be added explicitly

TRIUSTWORTHINESS AND AGENT ARCHITECTURES: IN SUMMARY

- From models that generate plausible text to systems that can be trusted when consequences matter
  - The technical bar changes from “high accuracy” to “reliable behavior under constraints”
- Two complementary ideas emerged
  - Trust is an engineering property: consistency, traceability, and accountability have to be designed in
  - Agency is an architectural property: capable behavior comes from explicit memory, control, and feedback loops, not a single model call
- A unifying scientific takeaway
  - Many modern “prompting methods” are rediscovering classical control patterns: retrieve, propose, evaluate, revise, act, learn
  - The difference between a demo and a dependable system is whether those patterns are explicit, testable, and constrained
- We will treat knowledge, rules/obligations, and verification as first-class components of an agent architecture

CAN KNOWLEDGE GRAPHS REDUCE HALLUCINATIONS IN LLMS ? HOW ?

Let us examine the following:

- Agrawal, G., Kumarage, T., Alghamdi, Z., & Liu, H. (2024, June). Can knowledge graphs reduce hallucinations in llms?: A survey. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 3947-3960).
  - A survey the characterizes the techniques

- Li, H., Appleby, G., Alperin, K., Gomez, S. R., & Suh, A. (2025). Mitigating LLM Hallucinations with Knowledge Graphs: A Case Study. *arXiv preprint arXiv:2504.12422*.
  - A system (with a primary focus on the intelligence analytics domain)

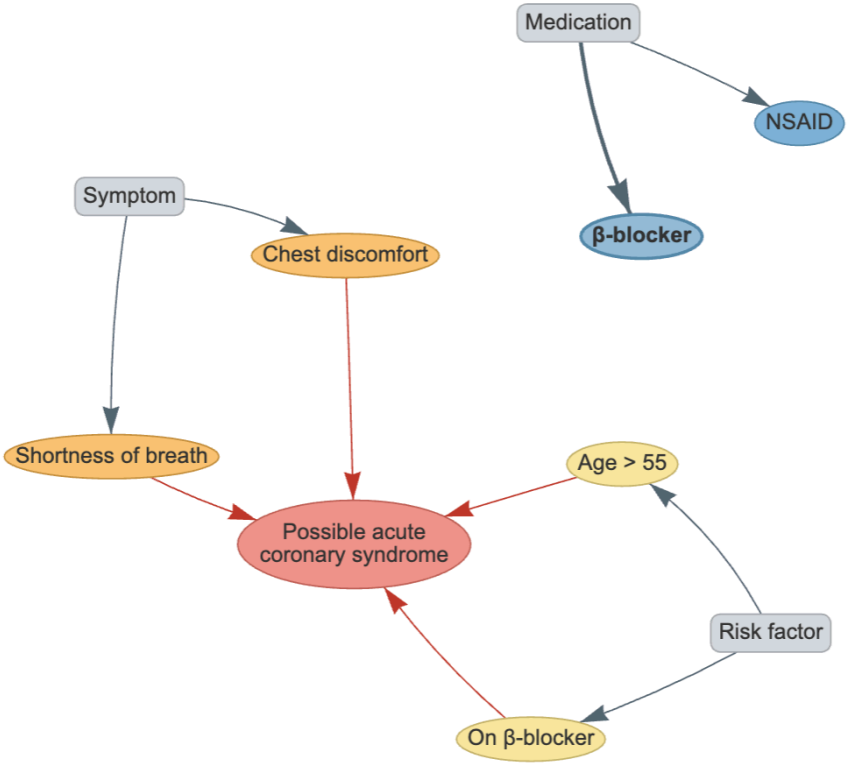
HALLUCINATIONS AREN'T A "BUG," THEY ARE A *SYSTEM PROPERTY*

- LLMs optimize next-token likelihood → fluent text, not guaranteed truth.
- In high-consequence domains, "mostly correct" is still unacceptable.
  - Healthcare: a single contraindication miss can harm a patient.
  - Compliance: one false "approved" decision can create legal exposure.
  - Cyber/IR: fabricated TTP or vulnerability links waste analyst time.
- Let us
  - Understand the main technique families for reducing hallucinations with Knowledge Graphs (KGs).
  - Know when each family helps, what it costs, and what new failure modes it introduces.

WHAT COUNTS AS "HALLUCINATION" (IN LLM OUTPUTS)

- Hallucination types (major)
  - **Factual hallucination**
    - Invented entity ("Study Z shows…")
    - Wrong attribute (dose, year, location, number)
  - **Relational hallucination**
    - Wrong link between entities (A caused B; A acquired B; drug treats disease)
  - **Omission / incompleteness**
    - Key constraint missing (contraindication, embargo, prerequisite)
    - Answer looks plausible but ignores a decisive rule
  - **Justification hallucination**
    - Correct final answer but fake reasoning path / fake citations / fake provenance
- Our aim: Eliminate / Reduce both false positives (made-up facts) AND false negatives (missing relevant facts).

WHAT *IS* A **KNOWLEDGE GRAPH** !



- A Knowledge Graph is a collection of ENTITIES and RELATIONSHIPS
- Long-term, curated, structured knowledge in the system

WHY KNOWLEDGE GRAPHS (KG) HELP

- KGs give **structure,** that text embeddings (by themselves) do not!
  - Typed entities & relations (what kinds of things exist; what links are allowed)
  - Composable multi-hop connectivity (support reasoning paths)
  - Constraints (schemas/ontologies/policies)
  - Provenance & timestamps (where/when a fact came from)
- But KGs don't "fix" the model by themselves
  - We must decide how the KG influences generation: retrieve vs reason vs constrain vs validate.
  - Incomplete or stale KGs shift errors from hallucinated facts → missing results / wrong queries.

SURVEY'S ORGANIZING IDEA: WHERE DO WE PLUG THE KG INTO THE PIPELINE?

- **1) Knowledge-aware inference** (runtime)
  - Use KG during answering; no retraining required.
- **2) Knowledge-aware training**
  - Pretraining / fine-tuning uses KG structure or KG-derived data to change model behavior.
- **3) Knowledge-aware validation**
  - Generate first, then verify claims against KG; repair if needed.

KNOWLEDGE-AWARE **INFERENCE**

- The **runtime use** of KGs
  - At inference time, the system *consults* the KG
    - So the LLM does not have to guess
  - No model weights change; the KG acts as external, structured memory and/or constraint.
- Three runtime families
  - KG-augmented **retrieval**
  - KG-augmented **reasoning** (iterative/multi-hop)
  - KG-controlled **generation** (tool/query mediated; schema constrained)

KG-AUGMENTED RETRIEVAL: THE KG-RAG FAMILY

- Core idea
  - Retrieve a small, relevant subgraph (facts + provenance) and put it into the prompt.
  - LLM's job becomes: summarize/compose from evidence, not invent.
- When it helps most
  - Missing factual grounding (names, dates, attributes)
  - Disambiguation (which 'Jaguar'? which 'Paris'?)
  - Reducing citation fabrication (if citations are retrieved as entities)
- Typical failure modes
  - Retriever misses a decisive fact → model fills the gap (hallucination)
  - KG incomplete/outdated → model answers confidently unless "unknown" handling exists

RETRIEVAL

- Example (clinical)
  - Question: “Is Drug X appropriate for Patient P today?”
  - Requires multiple facts: labs (eGFR), pregnancy status, current meds, contraindications, interactions.
- Why retrieval matters
  - Without evidence, the LLM may omit a single decisive constraint.
  - With evidence, the answer can explicitly cite which constraint blocks approval.

KG-AUGMENTED **REASONING** ITERATIVE, MULTI-HOP, CONSTRAINT SATISFACTION

- Core idea
  - Do not retrieve once. Alternate: reason → retrieve more → reason → …
  - Use intermediate sub-questions to drive which KG facts to fetch next.
- When it helps most
  - Multi-hop questions (A→B→C)
  - Multi-constraint decisions (“must satisfy A and B and not C”)
  - Producing an explanation trace (reasoning path over KG edges)
- New failure modes
  - Wrong decomposition (asks the wrong sub-question)
  - Search drift (the loop explores irrelevant paths)
  - False negatives: concludes “nothing found” because it didn’t traverse correctly

REASONING EXAMPLE: COMPLIANCE DECISION WITH MULTIPLE CHECKS

- Question
  - “Can shipment S proceed?”
- KG has facts about
  - Counterparty sanctions status
  - Item dual-use classification
  - License existence/validity
  - Destination embargo status
  - Route waypoints / restricted ports
- Reasoning path
  - Check sanctions → if true, stop (REJECT).
  - Else check embargo → if true, stop.
  - Else check dual-use+license → decide.
  - Output includes the first violated rule + evidence edges.

REASONING EXAMPLE: MULTI-HOP

- REASONING: Multi-hop “none exists” is valuable
- Question
  - “Which suppliers are 2 hops away from an embargoed region via subsidiaries?”
- Why LLMs hallucinate here
  - “Sounds plausible” connections get invented under pressure to answer.
- KG reasoning benefit
  - Either produce an explicit path Supplier→Holding→Region (with relations),
  - or confidently return ‘none found within depth=2’ with search constraints stated.

KG-CONTROLLED **GENERATION**

- Core idea: Constrain the model's degrees of freedom
  - Don't let the LLM directly emit factual claims.
  - Force it to: (a) write a structured query / tool call, (b) execute it, (c) summarize results.
- Common control patterns
  - Query-first answering (SPARQL/Cypher/API)
  - Schema/ontology-bound outputs (allowed fields + allowed relations)
  - "Logical drafts" + execution (draft rule evaluation → query KG for predicates)
- Why it reduces hallucination
  - Truth comes from executed KG results; the LLM becomes planner + narrator.

CONTROLLED GENERATION EXAMPLE

- Template : Natural Language → query → results → answer
- User asks
  - “What are the top 5 most common techniques used by APT groups?”
    - (APT: Advanced Persistent Threats)
- LLM is required to
  - Generate a query that counts Technique usage across APT entities
  - Execute it against the KG
  - Summarize the returned ranked list—no other facts allowed
- This shifts errors
  - From ‘invented facts’ → ‘query was wrong / incomplete’ (false negatives).

KG BASED **VALIDATION**

- Core idea: Generate, Verify, Repair*
  - Treat the LLM output as a draft; extract claims; check them against KG facts/rules.
  - Repair by: removing unsupported claims, asking clarification, or regenerating with constraints.
- When it helps most
  - High-stakes settings where wrong answers must be caught before delivery
  - Preventing fabricated citations and contradiction with known policies
- Key tradeoff
  - Adds compute + requires reliable claim extraction and mapping to KG predicates.

VALIDATION EXAMPLE: CONTRADICTION + CITATION HALLUCINATION

- Draft answer says
  - “Drug X is safe in pregnancy (Study Z, 2022).”
- KG checks reveal
  - Guideline: contraindicated in 1st trimester
  - Patient: 8 weeks pregnant
  - No publication node matching “Study Z, 2022”
- System response pattern
  - Block unsafe recommendation, cite the guideline node,
  - Ask for missing context if needed (trimester/indication),
  - Remove fake citation and propose supported references only.

TRAINING-TIME KG USE

- KG-aware **pretraining** (heavier)
  - Teach representations that respect entities/relations early.
  - Potentially improves factual recall and relational consistency—but expensive.
- KG-aware **fine-tuning** (lighter)
  - Domain adaptation: teach the model how to ask for evidence, format grounded answers, handle unknowns.
- Reality check
  - Training helps, but doesn't eliminate the need for runtime retrieval/ verification in high-consequence domains.

EVALUATION HIGHLIGHTS

- Runtime KG facts can rival scaling for small models
  - Reports >80% improvement in answer correctness in cited QA settings when adding KG facts (vs scaling).
- Reasoning-on-Graph (RoG) improvement
  - ChatGPT accuracy: 66.8% → 85.7% with KG reasoning augmentation.
- MindMap (clinical reasoning graph) outcome
  - Diagnosis + drug recommendation accuracy reported as 88.2%.
- Interpretation
  - Best gains appear when KG is used to *structure* retrieval/reasoning; not just appended as text.

CHALLENGES

- Coverage & freshness
  - KG may be incomplete; knowledge cutoff & updates remain practical issues.
- Grounding errors
  - Wrong entity linking / wrong relation selection can derail everything.
- Overconfidence
  - Systems must explicitly represent ‘unknown’ and avoid confident guesses when KG lacks evidence.
- Compute / latency
  - Iterative reasoning + validation improves fidelity but increases runtime.

LINKQ

LINKQ (CASE STUDY): THE "QUERY-MEDIATED ANSWERING" PATTERN

- Goal
  - Mitigate hallucinations by making the KG the factual source of truth.
  - LLM writes queries + explains; KG execution provides the facts.
- Core system stance
  - LLM is a planner + summarizer, not an authoritative memory store.
- Why this matters for our agents
  - This is a canonical example of KG-controlled generation + human-in-the-loop.

LINKQ WORKFLOW

- 1) Question refinement
  - Clarify intent so it maps cleanly to KG relations and constraints.
- 2) Grounding against KG schema
  - System helps resolve entity IDs / relation names so the LLM doesn’t invent schema.
- 3) Query generation
  - LLM produces a KG query (e.g., Wikidata query).
- 4) Human-in-the-loop review
  - User sees query/explanation; can edit before execution.
- 5) Execute + summarize results
  - Answer is derived from returned results; UI shows results table/graph.

LINKQ QUANTITATIVE EVALUATION

- Dataset
  - 120 complex questions (Mintaka / Wikidata): 5 types × 8 topics × 3 questions
  - Each question run 3 times; counted correct if succeeds at least once (handles LLM randomness).
- Baseline
  - Plain GPT-4 asked to directly generate Wikidata queries.
- Outcome
  - LinkQ substantially higher accuracy across all question types; runtime higher.

LINKQ RESULTS BY QUESTION TYPE (ACCURACY)

- Comparative
  - LinkQ 91.7% vs GPT-4 20.8%
- Yes/No
  - LinkQ 87.5% vs GPT-4 54.2%
- Generic
  - LinkQ 79.2% vs GPT-4 33.3%
- Multi-hop
  - LinkQ 75.0% vs GPT-4 16.7%
- Intersection
  - LinkQ 54.2% vs GPT-4 12.5%
- Interpretation
  - Intersection remains hardest: composing multiple conditions + joins is still brittle.

LINKQ QUALITATIVE FINDINGS: WHAT ERRORS REMAIN (AND WHY THEY MATTER)

- LinkQ reduces fabricated facts
  - Because answers come from executed KG results.
- But it can still fail via false negatives
  - Empty results even when data exists → query missed the right relations/joins.
- SME feedback (cyber KG)
  - Helpful for exploring APTs/techniques; surprising ability to form aggregate queries.
  - Desired improvements: better decomposition, alternative query candidates, richer entity descriptions.

DESIGN PLAYBOOK: CHOOSING A KG+LLM STRATEGY

- If the risk is ‘made-up facts’
  - Prefer controlled generation (query-mediated) + validation.
- If the risk is ‘missing a key constraint’
  - Prefer iterative reasoning (multi-check) + explicit unknown handling.
- If the task is mostly factual QA
  - Retrieval-first KG-RAG may be sufficient.
- If you need hard guarantees
  - Add rule execution (logic) + formal checks; treat LLM as planner/interface.

LINKQ SYSTEM

- https://github.com/mit-ll/linkq
- Demo mode
  - https://mit-ll.github.io/linkq/